

Contents

1	資料結構
1.1	BIT
2	字串
2.1	字典樹 trie
2.2	KMP
3	最短路徑
3.1	basic
4	圖論
4.1	凸包問題
5	數學
5.1	定理

1 資料結構

1.1 BIT

```

1 struct Binary_Indexed_Tree{
2     int n;
3     vector<int> bit;
4     void init(int _n){
5         n=_n+1;
6         bit=vector<int>(n,0);
7     }
8     int lowbit(int x) return x&-x;
9     void update(int x,int v){
10         for(;x<n;x+=lowbit(x)){
11             bit[x]+=v;
12         }
13     void range_update(int l,int r,int v){
14         update(l,v);
15         update(r+1,-v);
16     }
17     int qry(int x){
18         int ans=0;
19         for(;x>0;x-=lowbit(x)){
20             ans+=bit[x];
21         }
22         return ans;
23 };

```

2 字串

2.1 字典樹 trie

```

1 struct trie{
2     trie *nxt[26]={};
3     int cnt=0;
4     int sz=0;
5 };
6 trie *root=new trie();
7 void insert(const string &s){
8     trie *now=root;
9     for(auto i:s){
10         if(now->nxt[i-'a']==NULL){
11             now->nxt[i-'a']=new trie();
12         }
13         now=now->nxt[i-'a'];
14     }
15 int qry(string &s){
16     trie *now=root;
17     for(auto i:s){
18         if(now->nxt[i-'a']==NULL) return;
19         now=now->nxt[i-'a'];
20     }
21     return now->cnt;
22 }

```

2.2 KMP

```

1 vector<int> kmp(string s,string t){
2     if(t.size()>s.size()) return {};
3     vector<int> pi(t.size(),0);
4     for(int i=1,j=0;i<t.size();i++){
5         while(j>0 && t[j]!=t[i]){
6             j=pi[j-1];
7         }
8         if(t[j]==t[i]) j++;
9         pi[i]=j;
10    }
11    vector<int> ans;
12    for(int i=0,j=0;i<s.size();i++){
13        while(j>0 && t[j]!=s[i]){
14            j=pi[j-1];
15        }
16        if(t[j]==s[i]) j++;
17
18        if(j==t.size()){
19            ans.push_back(i-j+1);
20            j=pi[j-1];
21        }
22    }
23    return ans;
}

```

3 最短路徑

3.1 basic

```

1 // c++ code
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     // test comment
7     cout << "test string\n";
8 }

```

4 圖論

4.1 凸包問題

```

1 struct node{
2     int x,y;
3     bool operator<(const node &other) const{
4         return
5             (x<other.x) || (x==other.x && y<other.y);
6     };
7 };
8 struct ConvexHull{
9     vector<node> vec;
10    int n;
11    void init(vector<node> &v){
12        vec=v;
13        n=v.size();
14    }
15    int cross(node a,node o,node b){
16        return
17            (a.x-o.x)*(b.y-o.y)-(b.x-o.x)*(a.y-o.y);
18    }
19    vector<int> point;
20    vector<int> area;
21    void build(){
22        point=vector<int>(n+1);
23        area=vector<int>(n+1);
24        vector<node> up,down;

```

```

25     int upsum=0;
26     int downsum=0;
27     for(int i=0;i<n;i++){
28         while(up.size()>=2 &&
29             cross(up[up.size()-2],
30                 up[up.size()-1],
31                 vec[i])<=0){
32             upsum-=cross(up[up.size()-2],
33                         {0,0},up[up.size()-1]);
34             up.pop_back();
35         }
36         up.push_back(vec[i]);
37         if(up.size()>1)
38             upsum+=cross(up[up.size()-2],
39                         {0,0},
40                         up[up.size()-1]);
41
42         while(down.size()>=2 &&
43             cross(down[down.size()-2],
44                 down[down.size()-1],
45                 vec[i])>=0){
46             downsum-=cross(down[down.size()-2],
47                           {0,0},down[down.size()-1]);
48             down.pop_back();
49         }
50         down.push_back(vec[i]);
51         if(down.size()>1)
52             downsum+=cross(down[down.size()-2],
53                           {0,0},down[down.size()-1]);
54
55         area[i+1]=abs(upsum-downsum);
56
57         if(i+1==1) point[i+1]=1;
58         else point[i+1]=up.size()+down.size()-2;
59     }}
60
61     double getarea(int p){
62         return area[p]/2.0;
63     }
64     int getnum(int p){
65         return point[p];
66     }
67 };

```

5 數學

5.1 定理

- 錯排公式：(n 個人中，每個人皆不再原來位置的組合數)：

$$dp[0] = 1; dp[1] = 0;$$

$$dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]);$$

- 一個環上相鄰顏色不同的染色數量：

$$f(n, k) = (k - 1)^n + (-1)^n (k - 1)$$

n 是點的數量， k 是顏色的數量

- König 定理：

在二分圖中

最大匹配 = 最小點覆蓋

最小邊覆蓋 = 最大獨立集

點數 = 最大匹配 + 最小邊覆蓋

點數 = 最大獨立集 + 最小點覆蓋

[illegible]